

Arquitectura Hexagonal: cómo estructurar tu proyecto para que sea flexible y fácil de probar

¿Recuerdas los problemas que aparecieron en el proyecto de Streamlit cuando la API fallaba, querías cambiar la base de datos o probar los cálculos estadísticos sin conexión? Vamos a darle forma a una manera de organizar el código que soluciona esos dolores de cabeza y, además, te prepara para construir aplicaciones profesionales. Esta forma de trabajar se conoce como Arquitectura Hexagonal, o de Puertos y Adaptadores.

¿De dónde venimos? El problema del acoplamiento

En las primeras versiones de nuestro proyecto, todo estaba un poco mezclado: la función que pintaba la web también contenía llamadas a requests, el análisis estadístico accedía directamente a la base de datos, y los gráficos se rompían si la API no respondía. Cuando necesitábamos simular datos para probar, teníamos que comentar partes del código o parchear cosas, y un cambio en la estructura del JSON de la API externa nos obligaba a revisar archivos de lógica de negocio.

Esa situación se llama acoplamiento: tu lógica central (lo que la aplicación realmente hace) está atada a detalles como la conexión a internet, una librería concreta o una base de datos específica. La solución es construir un núcleo independiente, y luego enchufarle esos detalles como si fueran accesorios intercambiables.

La metáfora del enchufe y el núcleo

Imagina un hexágono que representa la esencia de tu aplicación. Dentro de él está la lógica de negocio pura: decidir cuándo pedir datos del clima, cómo procesarlos, cuándo guardarlos históricamente y cómo calcular estadísticas. Ese núcleo no sabe nada de internet, de bases de datos ni de interfaces de usuario.

Lo que el núcleo sí tiene son «puertos»: contratos que declaran lo que necesita recibir del exterior. Por ejemplo:

- Un puerto que diga «necesito obtener datos del clima para una ciudad».
- Otro puerto que diga «necesito guardar y recuperar información histórica».

Estos puertos son simplemente interfaces (en Python, clases abstractas) que establecen métodos claros, pero sin código concreto. Fuera del hexágono se conectan los «adaptadores»: piezas que implementan esas interfaces con tecnologías reales. Un adaptador podría usar la API de OpenWeather, otro podría leer un archivo JSON local; un adaptador podría usar SQLite, otro podría guardar en PostgreSQL. Desde el punto de vista del núcleo, todos son iguales mientras cumplan el contrato.

Lo crucial es la dirección de las dependencias: los adaptadores conocen al núcleo (importan sus interfaces), pero el núcleo no conoce a los adaptadores. El hexágono nunca hace `import requests` ni `import sqlite3`. Así, cambiar un adaptador por otro no supone tocar una sola línea del corazón de la aplicación.

Un ejemplo cotidiano

Piensa en tu teléfono móvil. En su interior está la lógica que sabe gestionar la carga de la batería. Ese circuito expone un puerto USB-C. Da igual que conectes el cargador de pared, un cable al coche, un power bank o el puerto USB de un ordenador. El teléfono no sabe qué hay al otro lado, solo necesita que lleguen los voltios adecuados según el estándar. Cada uno de esos cargadores es un adaptador que cumple el contrato USB-C. Si mañana inventan un cargador solar con USB-C, funcionará sin que tengas que abrir el teléfono a reprogramarlo.

Lo mismo ocurre con el software: si el núcleo expone un puerto «ObtenerClima», puedes enchufarle un adaptador que use Internet o uno que devuelva datos inventados para pruebas, y el resto del sistema no se entera.

Llevarlo a nuestro proyecto actual

Veamos cómo trasladar esta idea al código que ya habéis trabajado.

Identificar el dominio

¿Qué es lo que realmente hace nuestra aplicación, sin pensar en tecnologías? Algo así como: «Obtener los datos meteorológicos actuales de una ciudad, guardarlos en un histórico local y calcular estadísticas a partir de ese histórico». Eso será el interior del hexágono, y lo expresaremos como casos de uso (clases que representan acciones concretas).

Definir los puertos

Creamos dos interfaces (clases abstractas) en una carpeta llamada, por ejemplo, `dominio/puertos`:

- `ServicioClima`: con un método `obtener_datos(ciudad)` que promete devolver un diccionario con temperatura, humedad, etc.
- `RepositorioClima`: con métodos `guardar(ciudad, temperatura, humedad, fecha)` y `obtener_historico(ciudad, limite)`.

Estos puertos no contienen lógica, solo la firma de los métodos. El núcleo los usará, pero nunca sabrá quién está detrás.

Escribir los casos de uso

Un caso de uso es una clase que orquesta la lógica de negocio usando los puertos. Por ejemplo, la clase `ObtenerClimaActual` recibe en su constructor un `ServicioClima` y un `RepositorioClima`. Su método `ejecutar(ciudad)` intenta llamar a `servicio.obtener_datos(ciudad)`, extrae lo que necesita y lo guarda con el repositorio. Si el servicio falla, puede pedir al repositorio el último dato almacenado. Esta clase no hace ningún `import` de requests ni de SQLite; solo interactúa con las abstracciones.

Otro caso de uso, `GenerarEstadisticas`, recibiría únicamente el `RepositorioClima`. Pediría el histórico de una ciudad y aplicaría los cálculos con pandas o la librería que prefieras, devolviendo los resultados. Tampoco depende de cómo se hayan guardado los datos.

Construir los adaptadores

Ahora, fuera del núcleo, creamos las implementaciones concretas:

- **OpenWeatherAdapter**: implementa **ServicioClima**. Dentro sí usa **requests** y la URL de la API. Convierte el JSON a un diccionario común y lo devuelve.
- **SQLiteAdapter**: implementa **RepositorioClima**. Usa **sqlite3** para crear la tabla y ejecutar las consultas. El núcleo no sabe que es SQLite; solo ve el contrato.

Además, podemos crear un adaptador falso para pruebas: **ServicioFalso** que siempre devuelva una temperatura fija y una humedad fija, sin red ni dependencias reales.

Ensamblar todo en la interfaz de usuario

En el archivo principal de Streamlit (o en cualquier otro punto de entrada), creamos las instancias de los adaptadores y las inyectamos en los casos de uso:

```
from adaptadores.api.openweather_adapter import OpenWeatherAdapter
from adaptadores.persistencia.sqlite_adapter import SQLiteAdapter
from dominio.casos_de_uso.obtener_clima_actual import ObtenerClimaActual

servicio = OpenWeatherAdapter(api_key)
repositorio = SQLiteAdapter("clima.db")
caso_uso = ObtenerClimaActual(servicio, repositorio)

# Usarlo desde Streamlit:
datos = caso_uso.ejecutar("Madrid")
```

Observa que Streamlit solo importa los adaptadores y el caso de uso; no conoce la lógica interna de cómo se persiste ni cómo se consulta la API. El flujo es limpio y cualquier cambio en la tecnología externa afecta únicamente al punto de ensamblaje y al adaptador correspondiente.

Ejercicio mental: añadir una nueva fuente de datos

Supón que queremos obtener datos climáticos de otra API distinta, por ejemplo, WeatherAPI.com. ¿Qué habría que modificar dentro del núcleo? La respuesta es: nada. Solo necesitamos escribir un nuevo adaptador que implemente **ServicioClima**, y luego cambiar la línea de ensamblaje para usar **WeatherAPIAdapter** en lugar de **OpenWeatherAdapter**. La lógica de negocio ni se entera.

Otro escenario: imagina que queremos que la aplicación funcione sin internet usando un archivo JSON local como respaldo. Bastaría con crear **ServicioOffline** que lea el archivo y devuelva los datos con el mismo formato. Al inyectarlo en el caso de uso, el núcleo sigue igual. Esto hace que el software sea verdaderamente extensible.

Conceptos que emergen de esta práctica

Al trabajar con arquitectura hexagonal, os encontraréis con ideas muy valiosas:

- **Inyección de dependencias**: el caso de uso no crea sus propias dependencias; se las pasan desde fuera ya construidas. Así el control se invierte y podemos decidir qué implementaciones usar en cada momento (producción, pruebas, desarrollo).

- **Test unitario vs. test de integración:** Ahora la lógica de negocio se puede probar con adaptadores falsos (test unitario, muy rápido), mientras que los adaptadores reales se prueban con test de integración (por ejemplo, comprobar que OpenWeatherAdapter realmente llama a la API y devuelve algo). Antes esto era casi imposible porque todo estaba mezclado.
- **El punto de ensamblaje:** es ese fragmento de código donde se crean las instancias y se conectan los adaptadores a los casos de uso. Suele estar en el arranque de la aplicación y es el único sitio que conoce los adaptadores concretos.
- **Puertos primarios y secundarios:** Algunos llaman puertos primarios a los que inician la acción (por ejemplo, la interfaz de usuario que activa un caso de uso), y secundarios a los que el núcleo utiliza para obtener algo (APIs, bases de datos). En nuestra aplicación, Streamlit funcionaría como adaptador primario y OpenWeather y SQLite como secundarios.

¿Merece siempre la pena?

No hay que volverse dogmático. Si estás haciendo un prototipo pequeño que solo va a vivir una tarde, quizá no necesitas esta separación. Sin embargo, en nuestro proyecto ya hemos experimentado varios cambios: primero los datos venían directamente de la API, luego añadimos base de datos, luego quisimos hacer análisis... y cada paso rompía cosas. Ahora mismo, la inversión de separar el núcleo de los detalles nos dará una tranquilidad enorme para seguir añadiendo funcionalidades, probar sin miedo y colaborar sin pisarnos.

La arquitectura hexagonal no es un dibujo bonito ni una moda: es una herramienta mental para mantener tu código sano a medida que crece. Siéntete libre de empezar con un solo puerto y un par de adaptadores, y ampliar cuando el proyecto lo pida.

Con esta base, estáis listos para reescribir vuestro sistema de forma que el corazón de la aplicación quede protegido y los componentes externos sean simples accesorios intercambiables. El resultado será un software más fácil de mantener, de probar y de evolucionar.

Propuesta de arquitectura hexagonal

```

clima_hexagonal/
|
├─ app_streamlit.py           # Punto de entrada de la UI (adaptador
primario)
├─ config.py                 # Configuración: claves de API, rutas de
BD, etc.
|
├─ dominio/                  # Núcleo de la aplicación (no depende de
nada externo)
|   └─ __init__.py
|   |
|   └─ puertos/              # Puertos (interfaces) que expone el
dominio
|       └─ __init__.py
|       └─ servicio_clima.py  # Puerto: define el contrato para obtener
datos climáticos
|       └─ repositorio_clima.py # Puerto: define el contrato para guardar

```

```

y recuperar histórico
├── casos_de_uso/                                # Casos de uso (lógica de negocio
independiente)
│   ├── __init__.py
│   └── obtener_clima_actual.py                # Orquesta la obtención, guardado y
fallback
│   └── generar_estadisticas.py                # Calcula estadísticas a partir del
histórico
└── adaptadores/                                # Implementaciones concretas que se
enchufan a los puertos
    ├── __init__.py
    └── api/                                    # Adaptadores secundarios para fuentes de
datos externas
        ├── __init__.py
        └── openweather_adapter.py            # Implementa ServicioClima usando la API
de OpenWeather
        └── fake_servicio.py                  # Implementa ServicioClima con datos
falsos para pruebas
└── persistencia/                                # Adaptadores secundarios para
almacenamiento local
    ├── __init__.py
    └── sqlite_adapter.py                    # Implementa RepositorioClima con SQLite
└── presentacion/                                # Adaptador primario (opcional, si
queremos aislar Streamlit)
    ├── __init__.py
    └── streamlit_ui.py                      # Componentes de interfaz que usan los
casos de uso
└── tests/                                        # Pruebas unitarias y de integración
    ├── __init__.py
    └── test_obtener_clima_actual.py          # Prueba el caso de uso con adaptadores
falsos
    └── test_generar_estadisticas.py          # Prueba la lógica de estadísticas sin
dependencias reales

```